GPU Roofline Profiler: Debug Report

Olajide Badejo

July 23, 2026

Abstract

This is the honest account of what went wrong while building the profiler and how I fixed it, grouped by theme rather than by date. The dated raw material lives in `docs/ENGINEERING_LOG.md`; this document is the postmortem that draws the lessons out of it.

A theme runs through most of these. The failures that cost the most time were not the ones that produced an error message. They were the ones that produced a plausible looking number: a profiler that collected nothing and said nothing, a manifest that no parser could read, a metric that flattered the wrong thing, an optimisation that quietly targeted the wrong GPU architecture.

Contents

1 Environment and tooling

1.1 The machine had the GPU but none of the toolchain

The first probe of the target machine was a surprise in the honest direction. `nvidia-smi` reported a healthy RTX 5070 on driver 610.62 with a CUDA user mode driver at version 13.3, so the card and its runtime were fine. Everything needed to actually build against it was missing: no CUDA Toolkit and therefore no `nvcc`, no Visual Studio and therefore no `cl.exe` for `nvcc` to use as its host compiler, and neither Nsight Systems nor Nsight Compute. The machine had the gaming and display stack but not the developer stack.

The root cause was mundane: the tools were simply never installed. The right response was to resolve the toolchain deliberately rather than let large installers needing elevated permissions run unattended. In the meantime I built every part of the project that does not need a compiler, so the wait was not idle: the repository scaffold, the dash lint, the Python analysis package with its tests, and the report skeletons all came first.

1.2 The Python pins predated the interpreter

The machine's usable Python is CPython 3.14. The dependency versions I first pinned were older, and pip had no 3.14 wheels for them, so it fell back to building pandas from source with meson, which then failed for the same reason `nvcc` would have: no MSVC. The fix was to install the current releases that do ship 3.14 wheels and to re-pin `requirements.txt` to exactly those versions. Pinning to what actually installs and passes on this box is more honest than pinning to an aspiration that cannot be built here.

1.3 CUDA 13 removed the clock properties

A trivial smoke test would not compile: `cudaDeviceProp` has no member `clockRate`, and none named `memoryClockRate` either. CUDA 13 removed both. They were deprecated through 12.x and are now gone.

This mattered well beyond a smoke test, because the theoretical peak derivation needs the SM clock and the memory clock, so the entire ceiling calculation rested on two fields that no longer exist. The fix is to read them through `cudaDeviceGetAttribute`, which still supports them, and to check the returned status so that a future removal fails loudly rather than yielding a silent zero and a peak of zero.

The check that the replacement is right: 48 SMs at 128 lanes is 6144 CUDA cores, matching the vendor reference exactly, and at 2.625 GHz that gives 32.3 TFLOP/s against a vendor quote near 31. The 192 bit bus at 14.001 GHz doubled for the data rate gives 672.0 GB/s against a vendor quote of 672. Both landing on the reference numbers is what says the formulas are right.

1.4 The build silently targeted the wrong GPU

The first successful CMake configure printed `CUDA architectures: 75`. The card is sm_120.

I had set `CMAKE_CUDA_ARCHITECTURES` inside an `if(NOT DEFINED ...)` guard placed after `project()`. Enabling the CUDA language in `project()` installs a default of its own, so by the time my guard ran the variable was already defined and my detection never executed.

This is the quiet, expensive kind of bug. Nothing fails. The build just compiles for an older architecture than the card, and every performance number that followed would have been measured on code generated for the wrong target. The fix is to choose the architecture before `project()`.

I verified it afterwards with `cuobjdump`, which now reports `sm_120` SASS in every kernel object rather than trusting the configure message a second time.

1.5 A dependency that CMake 4 refuses to configure

`FetchContent` pulled yaml-cpp 0.8.0 and configure failed: compatibility with CMake below 3.5 has been removed, and this machine has CMake 4.3.1. I scoped a `CMAKE_POLICY_VERSION_MINIMUM` shim around that one dependency and unset it immediately after, so the compatibility relaxation applies to the fetched subproject and not to mine. Vendoring a patched copy would have been a maintenance burden for the same result, and hand writing a YAML parser to dodge one upstream version declaration would have been a poor trade.

1.6 NVTX broke the build through windows.h

Enabling NVTX turned a clean build into a syntax error in `timing.hpp`, a file that had not changed and that has nothing to do with profiling. `nvtx3/nvToolsExt.h` includes `windows.h`, which defines `min` and `max` as preprocessor macros, and the offending line is `std::max(1, std::min(wanted, max_launches))`. Once those are macros the expression stops parsing, and the error points at the victim rather than the culprit. Defining `NOMINMAX` next to the include that causes the problem fixes it and keeps the fix where the next person will look.

Alongside it, the CMake probe for NVTX tested `EXISTS` against `CUDAToolkit_INCLUDE_DIRS`, which is a *list* of two directories rather than one path, so it silently reported "NVTX not found" on a machine where the header was plainly present. `find_path` searches the list properly.

2 Measurement methodology

2.1 NVML explained a ceiling that looked impossible

The measured FP32 peak came out at 33.5 TFLOP/s against a theoretical 32.26, or 104 percent of a number nothing is supposed to exceed.

The NVML trace settles it. The log records an SM clock averaging 2857 MHz and peaking at 2865 MHz under load, while `cudaDevAttrClockRate` reports 2625 MHz. The runtime reports a nominal boost clock; the card boosts past it. Recomputing at the observed clock gives 35.2 TFLOP/s, and the measured figure is then 95 percent of it, exactly where a good FMA microbenchmark should land.

I changed nothing in the derivation. The theoretical ceiling stays derived from the clock the runtime reports, because that is the reproducible definition, and the report says plainly that the measured ceiling exceeds it and why. This is precisely the case the NVML monitor exists for: without the clock trace this would have looked like a counting bug and I would have gone hunting a factor of two that was never there.

2.2 Small kernels were measuring cache, not DRAM

SAXPY at 4 million elements reported 1583 GB/s of achieved bandwidth against a theoretical DRAM bandwidth of 672. At that size the two vectors total 32 MB and fit inside this card's 48 MB L2, so after warmup the kernel never touches DRAM and the figure is L2 bandwidth wearing a DRAM label. Past the cache the numbers behave: 560 to 566 GB/s at 16 and 64 million elements.

The lesson is about the byte count rather than the timing. An achieved bandwidth computed from a *theoretical* byte count assumes every byte comes from DRAM, and silently overstates DRAM

traffic when it does not. This is the gap the whole project is about, and it is why every point on the roofline is labelled with which byte count produced its intensity.

2.3 The shared memory tiling that did not pay

I expected the naive GEMM to be memory bound and tiling to fix it. Across every size measured, the tiled kernel is no faster than the naive one and at the smaller sizes it is slower.

The counters explain it. Naive and tiled report L2 hit rates equal to within measurement noise, and both move DRAM traffic at 5 to 8 GB/s against a 604 GB/s ceiling, so neither is close to memory bound. The naive kernel would move 68.7 GB if every operand read reached memory; it actually moves 80.8 MB, so cache absorbs better than 99.8 percent of its redundant loads. Tiling exists to remove redundant DRAM traffic and there is essentially none left to remove, while it still costs a synchronisation per tile step. The optimisation targets a bottleneck this hardware no longer has at this problem size.

What binds the naive kernel instead is instruction throughput: 98.5 percent of peak sustained SM throughput while achieving under 2 TFLOP/s of useful arithmetic. The fix is not to move data less far but to ask for it fewer times, which is what register blocking does, and that rung is nearly four times faster.

I record this as the most useful result in the project. It is the one place where measuring contradicted the textbook expectation, and the counters rather than the narrative decided it.

3 Kernel correctness

3.1 The tolerance was wrong, not the GEMM

Every GEMM variant failed its correctness test at every size: naive, tiled, register blocked, vectorized, and cuBLAS.

The clue that mattered was that all five reported the *identical* error to sixteen digits, and one of the five was cuBLAS. Five independent implementations do not share a bug. They do share a reference and an error metric.

My metric was a per element relative error, dividing each deviation by `max(|expected|, 1e-3)`. With random operands in $[-1, 1]$ each GEMM output is a sum of k signed products, so a good fraction of the entries land near zero through cancellation, and dividing an ordinary absolute error by one of those produces a huge ratio that measures the cancellation rather than the kernel.

Before changing anything I checked the arithmetic: a reported 3.2e-3 against the 1e-3 floor means an absolute error near 3.2e-6, while a 512 term sum of products of values in $[-1, 1]$ has magnitude around 13. That is a true relative error near 2.5e-7, which is fp32 epsilon. The kernels were right the whole time.

The fix was a normwise relative error, the largest absolute deviation over the largest magnitude in the reference, which is the standard way to check a GEMM and does not lose meaning near zero. I deliberately did not simply loosen the tolerance: that would have hidden the real problem behind a number chosen to make tests pass, and would then have been too loose to catch an actual bug at large k.

3.2 The counters found a bug in my own kernels

With the DRAM metrics finally collecting, the shared memory bank conflict counter read 134.5 million for the register blocked GEMM and 93.4 million for the vectorized one, against 40 thousand

for the tiled transpose.

I had applied the classic `[TILE][TILE + 1]` padding to the transpose tile, written a comment about why it matters, and then declared every GEMM shared tile unpadded. The register blocked inner loop walks down a column of its tile, so the address advances by a whole row per step; with an unpadded row length of 16 that stride is a multiple of the 32 bank width and the warp serialises on one bank. I had written the fix once and failed to apply it where it mattered most.

The fix carries a trap. The vectorized kernel reads its tile with a `float4`, so row starts must stay 16 byte aligned; padding by one float makes the row stride 260 bytes and turns an aligned vector load into undefined behaviour rather than a slow path. Padding by four floats preserves alignment and still moves each row off the colliding bank.

The measured outcome was not the clean sweep I expected. All 39 correctness tests still pass, and at 4096 the vectorized kernel gained 18 percent, from 8746 to 10325 GFLOP/s, while the register blocked and tiled kernels did not move outside run to run noise. Bank conflicts were the binding constraint in only one of the three. I am reporting that rather than implying the optimisation helped everywhere.

4 Profiling tooling

4.1 Nsight Compute collected nothing, convincingly

The first `ncu` run looked like a success. It attached to the process, reported the kernel by name with its grid and block dimensions, ran the kernel, and disconnected cleanly. Every metric value was `n/a`.

This is the most dangerous failure mode in the whole pipeline, because nothing errors. A CSV full of `n/a` parses perfectly well, and piped straight into the analysis it would have produced a report whose counter columns were empty for a reason nobody had noticed.

The cause is that on Windows the NVIDIA driver restricts GPU performance counters to administrators unless `RmProfilingAdminOnly` is set to 0 in the registry. The value was not set at all, so the driver default applied, and the shell was not elevated. This is the counter access caveat usually raised for WSL2, and it bites on native Windows too.

The wrapper script now checks for elevation up front and refuses with an explanation rather than producing a directory of `n/a`. I did not set the registry value: it is a permanent, system wide relaxation of who may read GPU counters, and that is the machine owner's decision to make deliberately rather than a side effect of my wanting a profile. Nsight Systems, worth noting, needs no elevation at all for CUDA and NVTX tracing.

4.2 The metric names in the documentation do not exist here

With elevation, most counters collected, and the two that mattered most did not: `dram__bytes_read.sum` and `dram__bytes_write.sum` still returned `n/a`. Those are the names in most published examples. On this chip the counters carry an `_op_` infix, and `ncu` does not reject an unsupported metric name: it accepts it, profiles the kernel, and reports `n/a`, producing a well formed CSV empty of the thing requested.

Reading `ncu --query-metrics` on the machine, rather than trusting documentation, gave the real names. The verified mapping is now recorded in `docs/profiling_guide.md` against the exact tool version, and the wrapper counts `n/a` values after every pass and warns with the offending metric names, so this specific silence can never happen twice.

4.3 Two ways to be wrong about a percentage

Folding the parsed metrics into per cell summaries, I aggregated everything the same way. Counters and percentages are not the same kind of quantity: profiling several launches and adding their byte counts gives the total traffic, which is what we want, but adding two L2 hit rates of 97 percent does not give 194 percent of anything. Extensive metrics now sum and intensive ones average.

That fix exposed a second thing, which is not a bug of mine. The naive GEMM reports an L2 sector hit rate of 100.39 percent. A hit rate is hits over accesses and cannot exceed 1. Re-running the same kernel gave 97.02, so it is noise in how the hardware attributes sector hits.

I chose to keep the value verbatim and flag it rather than clamp it to 100. Silently rewriting a measurement so it looks respectable is exactly the kind of quiet dishonesty this project exists to avoid, and a reader who sees 100.39 with a note attached learns something true about hardware counters that a tidy 100.00 would have hidden.

5 Report pipeline

5.1 Underscores in a placeholder path broke the compile

The main report guards every generated figure and table with a macro that prints a visible placeholder when the artifact does not exist yet, so the document compiles before and after the pipeline runs. The first version printed the missing path inside `\texttt`, and the paths contain underscores, which LaTeX reads as math subscripts in text mode and refuses to typeset. Wrapping the path in `\detokenize` renders them literally.

5.2 Every manifest was invalid JSON

Generating the environment appendix from the run manifest threw a JSON decode error: invalid escape. The driver wrote `"config_path": "configs\sweep.yaml"` straight into the document, and on Windows the path separator is a backslash, so `\s` is not a legal JSON escape and the whole file is malformed. The device name went out unescaped too.

This mattered more than it looks. The project's own rule is that results without a manifest do not exist, and an unparseable manifest is exactly as useless as a missing one. Every run up to that point carried one. Nothing had noticed because nothing had tried to read a manifest back until the report appendix needed it, and the file looked perfectly reasonable to a human eye.

The fix is a `json_escape` helper applied to every string written into the manifest. The wider lesson: hand rolling JSON with a stream operator is fine for a fixed schema right up until a value contains a character the format reserves.

5.3 Tables that ran off the page

Two layout faults, both of the kind that a compile without errors will happily ship. The timing summary is 62 rows, and a plain `tabular` is a single unbreakable box, so it ran off the bottom of the page and was clipped rather than continued. It is now a `longtable`, which repeats its header and marks continuations. The Nsight counter table has seven columns and ran off the right margin instead; it is scaled to the text width. The appendix tables carry raw metric names up to 49 characters with no break point, so they are set smaller rather than scaled, because a `longtable` spanning pages cannot be put inside a scaling box.

The general point is that LaTeX reports overfull boxes as warnings, not errors, so a report can compile cleanly and still be unreadable. Checking the log for overfull boxes is now part of judging whether a build is good, not just whether it finished.